

Projet Share++

Table des matières

1	Analyse préliminaire.....	3
1.1	Introduction.....	3
1.2	Objectifs.....	3
1.3	Planification initiale.....	4
2	Analyse / Conception.....	4
2.1	Concept.....	4
2.2	Stratégie de test.....	4
2.3	Risques techniques.....	4
2.4	Planification.....	4
2.5	Dossier de conception.....	5
3	Réalisation.....	5
3.1	Dossier de réalisation.....	5
3.2	Description des tests effectués.....	6
3.3	Erreurs restantes.....	6
3.4	Liste des documents fournis.....	6
4	Conclusions.....	6
5	Annexes.....	7
5.1	Résumé du rapport du TPI / version succincte de la documentation.....	7
5.2	Sources – Bibliographie.....	7
5.3	Journal de travail.....	7
5.4	Manuel d'Installation.....	7
5.5	Manuel d'Utilisation.....	7
5.6	Archives du projet.....	7

1 Analyse préliminaire

1.1 Introduction

Share++ est une solution de partage de fichiers sur un réseau local. Cette application est réalisée dans le cadre d'un projet informatique en développement d'application au CPNV à Ste-Croix. Ce projet vise principalement à simplifier le partage des fichiers d'utilisateurs entre ses différentes machines, sans avoir à passer par des services de cloud tiers ou l'envoi pièces jointes lourdes et pénibles pour se transférer des fichiers. Il garantit ainsi une meilleure confidentialité de ses données, ainsi qu'une vitesse de transfert maximale car elle ne dépend que du réseau (pas de OneDrive ou de couche supplémentaires qui ajoutent de la complexité). Aucun travail préalable n'a été effectué sur ce projet spécifique en amont de la période de réalisation.

Contrairement aux informations présentes dans le descriptif du projet du cahier des charges, la partie redirection de port et Internet ne sera pas traitée, en raison des restrictions du CPNV vis-a-vis de la redirection de port.

1.2 Objectifs

Les objectifs principaux de Share++ sont repris directement depuis le cahier des charges :

- Sélectionner un dossier à partager via une interface graphique ou via le menu contextuel Windows (clic droit).
- Démarrer/arrêter un serveur HTTP local.
- Générer un lien d'accès et un QR Code pour faciliter la connexion depuis un autre appareil (PC, tablette, smartphone).
- Afficher en temps réel les logs de connexion (ex. : "Utilisateur X a téléchargé Fichier.pdf").
- Copier automatiquement l'URL du serveur (pour faciliter la configuration de la redirection de port.)

Le serveur génère une page web simple listant les fichiers disponibles, avec des liens de téléchargement directs. L'application est compilée en exécutable autonome, ne nécessitant aucune installation de Python sur la machine cible.

1.3 Planification initiale

Le projet débute le 2 février 2026 et dure jusqu'au 31 mars 2026. Il est découpé en 5 sprints :

	02.02	11.02	18.02	25.02	04.03	11.03	18.03	25.03	31.03
Sprint 1 : Analyse									
Sprint 2 : Maquettes, schémas et conception									
Sprint 3 : Code									
Sprint 4 : Sécurité et intégration système									
Sprint 5 : Tests, Documentation									

Ce chapitre montre la planification du projet. Celui-ci peut être découpé en tâches qui seront planifiées. Il s'agit de la première planification du projet, celle-ci devra être revue après l'analyse. Cette planification sera présentée sous la forme d'un diagramme.

Ces éléments peuvent être repris des spécifications de départ.

2 Analyse / Conception

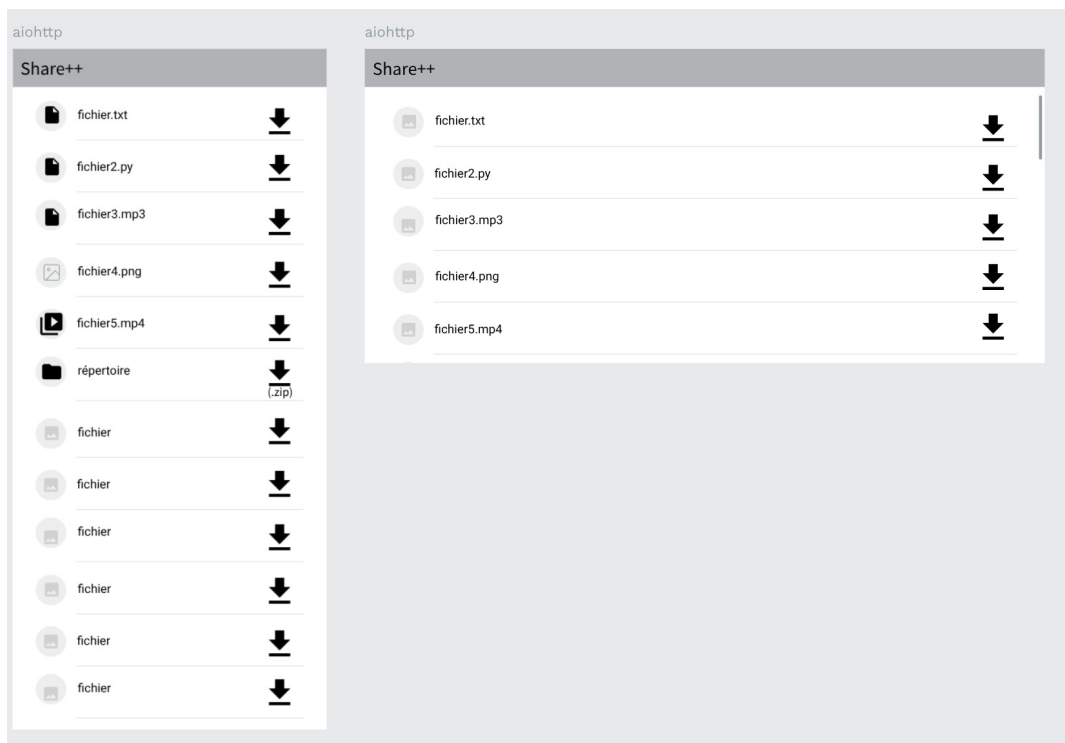
2.1 Concept

Analyse fonctionnelle : Le cas d'utilisation principal de l'application est de permettre, lorsqu'un utilisateur sélectionne un dossier local, de l'ouvrir via le menu contextuel windows (ou directement depuis l'application), pour ensuite démarrer un serveur de fichier, et de pouvoir y accéder facilement en scannant un code qr généré automatiquement, ou en inscrivant l'url complet dans un navigateur. L'utilisateur pourra ensuite choisir les fichiers ou les répertoires qu'il souhaite télécharger.

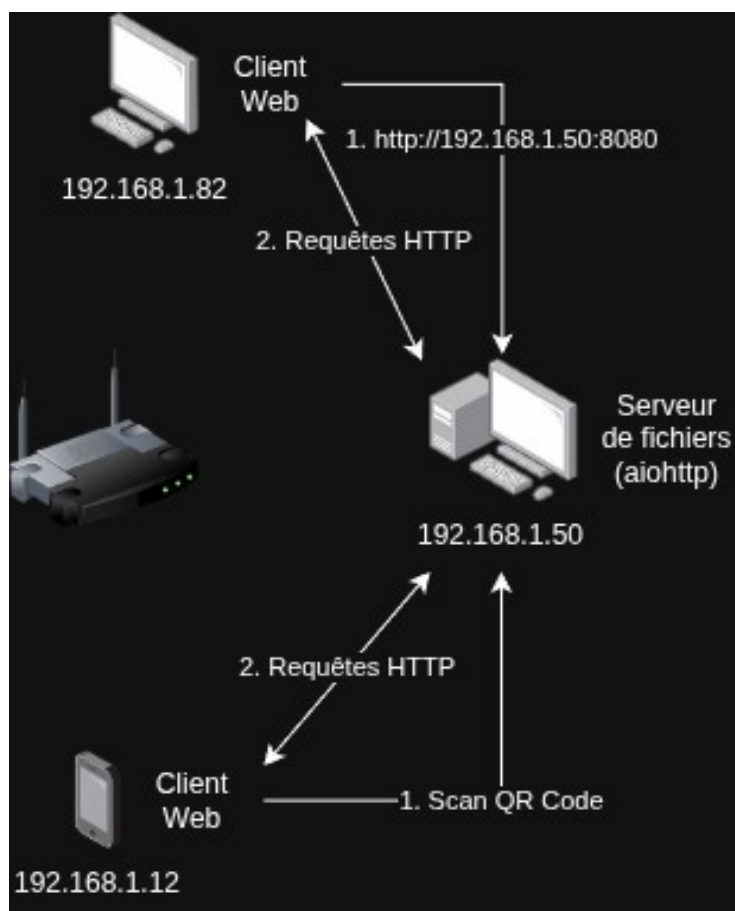
Voici la maquette de l'application



Voici la maquette de l'interface de téléchargement



Voici le schéma réseau :



2.2 Stratégie de test

La stratégie de test vise à valider le fonctionnement du transfert, la sécurité

Types de tests :

1. Tests unitaires : Vérifier l'utilisation du JSON et la génération d'URL
2. Tests d'intégration : Vérifier que le serveur de fichier aiohttp est bien configuré par rapport aux paramètres rentrés depuis l'application Flet.
3. Tests End-to-End : Scan du QR code avec un smartphone, et téléchargement de fichiers

Moyens de mise en œuvre des tests :

- Automatisation des tests unitaires et d'intégration grâce à la librairie pytest.
- L'environnement de test correspond au matériel utilisé pendant la réalisation : Un laptop personnel Lenovo Yoga sur Fedora 43 (Linux), comme serveur, et un smartphone Android (realme neo7) comme client pour accéder à l'interface de téléchargement.
- Infrastructure réseau : Utilisation du Hotspot du smartphone pour créer un sous-réseau, non-restreint par les restrictions du réseau CPNV.

Couverture et données de test :

Les tests ne sont pas exhaustifs. Ils testent les fonctionnalités principales de l'application, et garantissent la sécurité. Les données utilisées seront un fichier .txt pour tester le téléchargement direct, ainsi qu'un répertoire incluant un fichier .txt afin de tester la compression en .zip avant l'envoi.

Décrire la stratégie globale de test:

- *types de tests et ordre dans lequel ils seront effectués.*
- *les moyens à mettre en œuvre.*
- *couverture des tests (tests exhaustifs ou non, si non, pourquoi ?).*
- *données de test à prévoir (données réelles ?).*
- *les testeurs extérieurs éventuels.*

2.3 Risques techniques

Risque 1 : Le pare-feu Windows pourrait bloquer les requêtes entrantes sur le port 8080

Solution : Documenter la notion de port, laisser l'utilisateur spécifier le port s'il le souhaite

Risque 2 : Accès simultané : Plusieurs téléchargement en même temps pourraient faire crasher un serveur pas asynchrone

Solution : Utilisation d'une librairie asynchrone (aiohttp) pour gérer plusieurs téléchargements.

Risque 3 : Instance multiple : Si l'utilisateur lance plusieurs fois l'application et qu'il essaie de créer des serveurs simultanément, l'application pourrait crasher.

Solution : Implémenter une détection d'instance qui tournerait déjà au lancement de l'application

- *risques techniques (complexité, manque de compétences, ...).*

Décrire aussi quelles solutions ont été appliquées pour réduire les risques (priorités, formation, actions, ...).

2.4 Planification

Révision de la planification initiale du projet :

- *planning indiquant les dates de début et de fin du projet ainsi que le découpage connu des diverses phases.*
- *partage des tâches en cas de travail à plusieurs.*

*Il s'agit en principe de la planification **définitive du projet**. Elle peut être ensuite affinée (découpage des tâches). Si les délais doivent être ensuite modifiés, le responsable de projet doit être avisé, et les raisons doivent être expliquées dans l'historique.*

2.5 Dossier de conception

Choix matériel/software :

Le projet est codé en python (3.14), car c'est un langage très utilisé et qui supporte beaucoup de bibliothèques bien maintenues, et qui sont justement utilisés dans ce projet :

- Flet, la bibliothèque python cross plateforme pour l'interface de l'application, rendant ainsi possible d'étendre la compatibilité de l'app dans le futur.
- aiohttp, une bibliothèque de serveur HTTP asynchrone pour le serveur de fichier, capable sur le papier de traiter le risque d'accès simultanés au serveur.
- qrcode, une bibliothèque python de génération de code QR.

Architecture du programme : J'ai décidé d'adopter une architecture MVC (Model View Controller) pour mon projet. Je le trouve adapté à mon projet et pense pouvoir séparer les fonctionnalités comme ceci : Le Modèle gère la création du serveur asynchrone, la persistance des données, les logs et l'installation au menu contextuel Windows, View contient l'interface de l'application Flet, et de la page web de téléchargement. Controller gère les clics des boutons dans l'application. Le fichier main.py est utilisé comme point d'entrée et initialise l'application.

Fournir tous les documents de conception:

- *le choix du matériel HW*
- *le choix des systèmes d'exploitation pour la réalisation et l'utilisation*
- *le choix des outils logiciels pour la réalisation et l'utilisation*

- *site web*: réaliser les maquettes avec un logiciel, décrire toutes les animations sur papier, définir les mots-clés, choisir une formule d'hébergement, définir la méthode de mise à jour, ...
- *bases de données*: décrire le modèle relationnel, le contenu détaillé des tables (caractéristiques de chaque champs) et les requêtes.
- *programmation et scripts*: organigramme, architecture du programme, découpage modulaire, entrées-sorties des modules, pseudo-code / structogramme...

Le dossier de conception devrait permettre de sous-traiter la réalisation du projet !

3 Réalisation

3.1 Environnement

- OS : Windows 11 (ordinateur CPNV) / Fedora 43 (laptop personnel)
- Matériel : Laptop personnel Lenovo Yoga, Smartphone Android Realme Neo7

3.2 Structure du projet

La structure est également représentée plus succinctement dans le README du projet
Sharepp/

```
├─ docs # Dossier de projet, journaux, maquettes, schémas...
│   └─ Dossier_de_projet_Share++_Nathan2.pdf
│   └─ Dossier_de_projet_Share++_Nathan.odt
│   └─ Dossier_de_projet_Share++_Nathan.pdf
│   └─ Journal_de_bord_Share++_Nathan.pdf
│   └─ Journal_de_test_Share++_Nathan.pdf
│   └─ Journal_de_travail_Share++_Nathan.ods
│   └─ Journaux.pdf
│   └─ maquette_app.pdf
│   └─ maquette_app.png
│   └─ maquette_web.pdf
│   └─ maquette_web.png
│   └─ reseau.drawio
│   └─ schema_reseau.drawio
│   └─ schema_reseau.png
│   └─ Sprint_planning.md
├─ LICENSE
├─ pytest.ini
├─ README.md # Instructions d'installation
├─ requirements.txt # Dépendances
└─ src
    └─ assets
        └─ icons
    └─ controllers
        └─ app_controller.py # Logique métier
```

```
|— main.py # Point d'entrée de l'application
|— models
|   |— model.py # Gestion des données persistentes
|— report.html
|— settings.json # Paramètres de l'application
|— tests
|   |— test_controller.py
|   |— test_model.py
|— views
|   |— download_view.py # Template HTML pour aiohttp
|   |— view.py # Interface de l'application Flet
```

3.3 Fonctionnalités techniques importantes

Serveur HTTP

- basé sur aiohttp
- asynchrone permettant la gestion multi-clients

Compression des répertoires

- zip des dossiers avant l'envoi

QR Code

- Génération du QR Code en base 64 pour l'affichage direct sur flet

3.4 Dossier de réalisation

Cette section détaille l'implémentation physique du projet Share++, les outils utilisés et la structure finale du projet.

Environnement de développement :

- Ordinateurs : Ordinateur fourni par le CPNV (windows 11), laptop personnel
- Système d'exploitation : Windows 11, Fedora 43 (Linux)
- Langage : Python 3.14.2
- Version : 1.0
- Bibliothèques supplémentaires :

Librairie	Version	Utilité
flet	0.80.5	Framework d'interface utilisateur pure python cross-plateform
qrcode	8.2	Génération de codes QR pour le partage d'URL vers smartphone
aiohttp	3.13.3	Serveur web asynchrone pour traiter les téléchargements
Pillow	11.3.0	Support pour le traitement d'images sur python

Structure du projet :

L'arborescence est aussi présente dans le README du dépôt.

Modèle de paramètres :

Le fichier `settings.json`, situé dans le dossier `src/`, permet de sauvegarder la configuration de l'utilisateur. Voici sa structure :

```
{
  "selected_path": "/path/to/...", // Chemin du dossier partagé
  "port": <port>, // Port d'écoute du serveur (défaut=8080)
  "is_protected": true/false, // Activation de la sécurité
  "password": "", // Mot de passe (optionnel)
  "logs_visible": true, // État de l'affichage de la console
  "qr_enabled": true, // Génération du QR Code activée
  "remember_path": false // Mémorisation du chemin au lancement
  "remember_path": false // Mémorisation du chemin au lancement
}
```

Décrire la réalisation "physique" de votre projet

- les répertoires où le logiciel est installé
- la liste de tous les fichiers et une rapide description de leur contenu (des noms qui parlent !)
- les versions des systèmes d'exploitation et des outils logiciels
- la description exacte du matériel
- le numéro de version de votre produit !
- programmation et scripts: librairies externes, dictionnaire des données, reconstruction du logiciel - cible à partir des sources.

NOTE : Évitez d'inclure les listings des sources, à moins que vous ne désiriez en expliquer une partie vous paraissant importante. Dans ce cas n'incluez que cette partie...

3.5 Description des tests effectués

Cette section détaille la mise en pratique de la stratégie de test. Les premiers tests de l'application ont été réalisés le 10 mars 2026.

Conditions exactes des tests

Pour éviter les restrictions et les problèmes potentiels liés au réseau du CPNV, les tests ont été effectués sur un sous-réseau via partage de connexion du smartphone. Voici les appareils utilisés et leurs utilité :

- Serveur : Laptop Lenovo Yoga (Fedora 43)
- Client : Smartphone Realme Neo7 (Android)

Un environnement précis a été utilisé pour se protéger des problèmes éventuels liés à l'infrastructure réseau restrictive du CPNV. Les détails de l'environnement, le plan des tests, ainsi que des rapports de tests sont présents dans les annexes.

Pour chaque partie testée de votre projet, il faut décrire:

- *les conditions exactes de chaque test*
- *les preuves de test (papier ou fichier)*
- *tests sans preuve: fournir au moins une description*

3.6 Erreurs restantes

Les fonctionnalités principales de l'application sont fonctionnels comme documenté dans le journal des tests. Tous les problèmes bloquant rencontrés ont été traités, mais des défauts mineurs persistent :

1. Authentification à la page de téléchargements
 - Description : Lors de la connexion à la page, l'utilisateur doit obligatoirement rafraîchir sa page pour accéder à l'authentification en cas de partage protégé par mot de passe. De plus, le champ « username » est présent par défaut et ne peut pas être retiré facilement. Il est ignoré lors de la requête, seul le champ mot de passe est traité.
 - Conséquences sur l'utilisation du produit : L'expérience utilisateur est dégradée par ces actions et réflexions inutiles. L'utilisateur s'attend à ce que cette étape soit plus intuitive.
 - Actions envisagées : Remplacer l'implémentation de l'authentification par une page de connexion HTML-CSS classique, utilisant un Token expirable.
2. Interface de la page de téléchargements
 - Description : L'interface de téléchargement est trop basique et ne permet aucune autre actions que le téléchargement direct des items présents.
 - Conséquences sur l'utilisation du produit : L'utilisateur peut être frustré de ne pas pouvoir effectuer des actions basiques comme sélectionner plusieurs items avant de lancer le téléchargement, prévisualiser le fichier qu'il s'apprête à télécharger, entrer dans les répertoires présents dans la page, pour ne pas avoir à forcément télécharger le répertoire entier.
 - Actions envisagées : Améliorer l'interface, ainsi que changer les fonctions de logiques afin d'être plus permissives sur certains points (support de téléchargement multiples, sécurité anti accès transversal par l'URL (autoriser la lecture en descendant, mais empêcher de remonter.)

S'il reste encore des erreurs:

- *Description détaillée*
- *Conséquences sur l'utilisation du produit*
- *Actions envisagées ou possibles*

3.7 Liste des documents fournis

Les éléments suivant constituent la livraison finale de mon projet Share++. Tous les documents sont en version 1.0, version de livraison.

Document	Format	Description
Dossier de projet	PDF	Document principal, comprenant l'analyse, la conception et la réalisation.

Lister les documents fournis au client avec votre produit, en indiquant les numéros de versions

- le rapport de projet
- le manuel d'Installation (en annexe)
- le manuel d'Utilisation avec des exemples graphiques (en annexe)
- autres...

4 Conclusions

Le projet Share++ semble répondre à majorité des objectifs fixés dans le cahier des charges. Les fonctionnalités principales ont été implémentées avec succès :

- Application windows permettant de sélectionner un dossier à partage sur une interface graphique
- Démarrage d'un serveur HTTP asynchrone localement
- Génération de l'URL de connexion, ainsi qu'un code QR pour l'accès simplifié pour mobile
- Accès au fichiers grâce à une page de téléchargement accessible depuis un navigateur, fonctionnant donc sur tous les appareils.
- Affichage des logs de connexions

Globalement, les objectifs principaux du projet sont atteints, avec une application fonctionnelle et utilisable dans un contexte réel.

Points positifs :

- Architecture claire, reprenant le modèle MVC
- Utilisation de technologies modernes et adaptées au projet (aiohttp, flet)
- Interface d'application claire, ergonomique, simple d'utilisation
- Bon fonctionnement des téléchargements en réseau local en termes de performances et de rapidité.
- Gestion multi-clients grâce à asynchrone correctement traitée

Points négatifs :

-

Développez en tous cas les points suivants:

- *Objectifs atteints / non-atteints*
- *Points positifs / négatifs*
- *Difficultés particulières*
- *Suites possibles pour le projet (évolutions & améliorations)*

5 Annexes

5.1 Résumé du rapport du TPI / version succincte de la documentation

5.2 Sources – Bibliographie

Liste des livres utilisés (Titre, auteur, date), des sites Internet (URL) consultés, des articles (Revue, d'Analyse / Conception)

5.3 Concept

Le concept complet avec toutes ses annexes:

Par exemple :

- *Multimédia: carte de site, maquettes papier, story board préliminaire, ...*
- *Bases de données: interfaces graphiques, modèle conceptuel.*
- *Programmation: interfaces graphiques, maquettes, analyse fonctionnelle...*
- *...*

5.4 Stratégie de test

Décrire la stratégie globale de test:

- *types de des tests et ordre dans lequel ils seront effectués.*
- *les moyens à mettre en œuvre.*
- *couverture des tests (tests exhaustifs ou non, si non, pourquoi ?).*
- *données de test à prévoir (données réelles ?).*
- *les testeurs extérieurs éventuels.*

5.5 Risques techniques

- *risques techniques (complexité, manque de compétences, ...).*

Décrire aussi quelles solutions ont été appliquées pour réduire les risques (priorités, formation, actions, ...).

5.6 Planification

Révision de la planification initiale du projet :

- *planning indiquant les dates de début et de fin du projet ainsi que le découpage connu des diverses phases.*

- *partage des tâches en cas de travail à plusieurs.*

*Il s'agit en principe de la planification **définitive du projet**. Elle peut être ensuite affinée (découpage des tâches). Si les délais doivent être ensuite modifiés, le responsable de projet doit être avisé, et les raisons doivent être expliquées dans l'historique.*

5.7 Dossier de conception

Fournir tous les document de conception:

- *le choix du matériel HW*
- *le choix des systèmes d'exploitation pour la réalisation et l'utilisation*
- *le choix des outils logiciels pour la réalisation et l'utilisation*
- *site web: réaliser les maquettes avec un logiciel, décrire toutes les animations sur papier, définir les mots-clés, choisir une formule d'hébergement, définir la méthode de mise à jour, ...*
- *bases de données: décrire le modèle relationnel, le contenu détaillé des tables (caractéristiques de chaque champs) et les requêtes.*
- *programmation et scripts: organigramme, architecture du programme, découpage modulaire, entrées-sorties des modules, pseudo-code / structogramme...*

Le dossier de conception devrait permettre de sous-traiter la réalisation du projet !

6 Réalisation

6.1 Dossier de réalisation

Décrire la réalisation "physique" de votre projet

- *les répertoires où le logiciel est installé*
- *la liste de tous les fichiers et une rapide description de leur contenu (des noms qui parlent !)*
- *les versions des systèmes d'exploitation et des outils logiciels*
- *la description exacte du matériel*
- *le numéro de version de votre produit !*
- *programmation et scripts: bibliothèques externes, dictionnaire des données, reconstruction du logiciel - cible à partir des sources.*

NOTE : Évitez d'inclure les listings des sources, à moins que vous ne désiriez en expliquer une partie vous paraissant importante. Dans ce cas n'incluez que cette partie...

6.2 Description des tests effectués

Pour chaque partie testée de votre projet, il faut décrire:

- les conditions exactes de chaque test
- les preuves de test (papier ou fichier)
- tests sans preuve: fournir au moins une description

6.3 Erreurs restantes

S'il reste encore des erreurs:

- Description détaillée
- Conséquences sur l'utilisation du produit
- Actions envisagées ou possibles

6.4 Liste des documents fournis

Lister les documents fournis au client avec votre produit, en indiquant les numéros de versions

- le rapport de projet
- le manuel d'Installation (en annexe)
- le manuel d'Utilisation avec des exemples graphiques (en annexe)
- autres...

7 Conclusions

Développez en tous cas les points suivants:

- Objectifs atteints / non-atteints
- Points positifs / négatifs
- Difficultés particulières
- Suites possibles pour le projet (évolutions & améliorations)

8 Annexes

8.1 Résumé du rapport du TPI / version succincte de la documentation

8.2 Sources – Bibliographie

Liste des livres utilisés (Titre, auteur, date), des sites Internet (URL) consultés, des articles (Revue, date, titre, auteur)... Et de toutes les aides externes (noms)

8.3 Journal de travail

Date	Durée	Activité	Remarques

8.4 Manuel d'Installation

8.5 Manuel d'Utilisation

8.6 Archives du projet

Media, ... dans une fourre en plastique